

Sistemi Operativi

Corso di Laurea in Informatica

Sapienza Università di Roma

a.a. 2026/2027

Marco Raoul Marini

marini@di.uniroma1.it

Gabriele Tolomei

tolomei@di.uniroma1.it

Lecture 1.2: Introduction to OS & Hardware Support

The Philosophy of the Kernel

"An operating system is a program that acts as an intermediary between a user of a computer and the computer hardware."

— Avi Silberschatz

The OS must make the system convenient to use, efficient in resource allocation, and completely secure from application interference.

| Lecture Roadmap

This lecture transitions from hardware foundations to the software that drives them, detailing the hardware support required for operating systems.

It is organized into **7 major sections**:

1. **Operating System Overview & History** — where we came from and why
2. **Core OS Abstractions** — virtualizing resources (Processes, Address Spaces, Files)
3. **Hardware Support & Dual-Mode Execution** — keeping user programs in check
4. **The System Call Interface (Syscalls)** — requesting kernel services
5. **Interrupts, Exceptions & Traps** — the mechanics of hardware/software event handling
6. **Memory Protection & DMA Coordination** — protecting the physical address space
7. **Summary & Path to Virtualization** — synthesis

Let's define the Operating System.

§ 1 — Operating System Overview & History

| What is an Operating System?

An **Operating System (OS)** is a collection of system software that manages hardware resources and provides common services for application programs.

- It is the first layer of software loaded when a computer boots.
- It runs in a highly privileged execution mode (Kernel Mode).
- **Core Paradox:** A good OS should be almost invisible to the user. You only notice it when it fails, runs out of memory, or crashes.

| View 1: The OS as an Abstraction Layer

Without an OS, application programmers would have to write custom code to:

- Control disk drive read heads sector-by-sector.
- Manage raw physical RAM addresses directly.
- Send bit patterns directly to physical graphics ports.

The OS solution: It presents a clean, abstract, virtual machine to applications.

- Sector numbers become **Files** and **Directories**.
- Raw memory becomes a private **Virtual Address Space**.
- Network interfaces become **Sockets**.

| View 2: The OS as a Resource Manager

Computing resources (CPUs, RAM, Disk, I/O devices) are physical, expensive, and shared.

The OS acts as the **manager** of these resources:

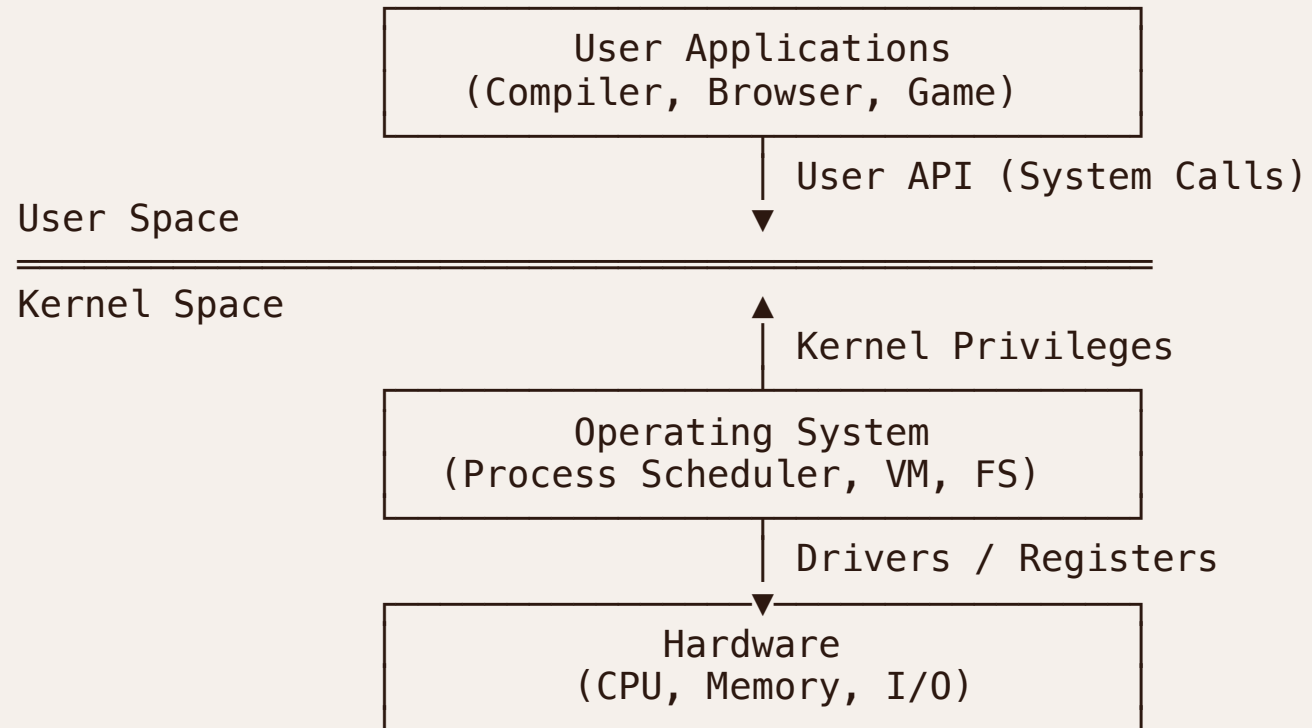
- **Resource Allocation:** Decides which program gets to use the CPU, when, and for how long.
- **Fairness:** Ensures no application starves other applications of resources.
- **Efficiency:** Optimizes resource usage to maximize overall throughput and minimize latency.

| View 3: The OS as a Control Program

An operating system acts as an authority that governs program execution.

- **Isolation:** Prevents application programs from interfering with each other (e.g., App A cannot read or write to App B's memory).
- **Protection:** Prevents application programs from modifying kernel code or settings.
- **Error Control:** Catches application crashes (like division by zero or null pointer access) and terminates the offending program safely without bringing down the entire machine.

OS Architectural Placement



| History: 1. Direct Access Systems (1940s-1950s)

- **Hardware:** Vacuum tubes, plugboards. No operating system.
- **Programming:** Write binary machine code directly onto paper tape or punch cards.
- **Execution:** A single programmer books the physical console for a slot.
 - They load their punch cards, run the code, and debug it on the spot using lights and switches.
- **Inefficiency:** The expensive CPU remains idle while the programmer loads cards, mounts tapes, or thinks.

| History: 2. Batch Processing Systems (Late 1950s)

To keep the CPU busy, operators batched jobs together:

- **Resident Monitor:** The first primitive OS. A small program that sits permanently in memory.
- **Execution Flow:**
 - Programmer submits punch cards to an operator.
 - Operator batches jobs onto magnetic tape.
 - Resident Monitor loads the first job, runs it, and automatically loads the next job when it finishes.
- **Problem:** I/O operations (reading cards/writing to printer) are extremely slow compared to CPU speed. The CPU still idles waiting for tape reads.

| History: 3. Multiprogramming Systems (1960s)

To eliminate I/O idle time, multiple jobs were loaded into memory simultaneously:

Memory Layout:

Resident Monitor	
Job 1	(Waiting for Disk Read)
Job 2	(Running on CPU)
Job 3	(Waiting for Printer)

- When Job 1 blocks on an I/O operation, the OS switches the CPU to run Job 2.
- **Required Hardware Support:** Memory protection (to prevent Job 1 from writing to Job 2's space) and CPU scheduling.

| History: 4. Timesharing Systems (1970s)

Timesharing (e.g., Multics, UNIX) introduced interactive computing:

- **Concept:** The CPU quickly cycles through active users using a timer interrupt (Round-Robin).
- **User Experience:** Each user interacts with the system via a terminal. Because the CPU switches between users in milliseconds, it feels like they have the entire machine to themselves.
- **New Challenges:** Concurrency issues, file access control, virtual memory, and process isolation.

| History: 5. Personal Computing to Cloud Era

- **1980s:** Personal computers (PCs) like MS-DOS. Initially lacked hardware protection, sacrificing security for affordability.
- **1990s-Present:** High-performance desktop and server systems (Windows NT, Linux, macOS). Multi-user, multi-tasking, fully protected.
- **Mobile Era (2000s-Present):** iOS, Android. Focus on power efficiency, sandboxing, and security.
- **Virtualization & Cloud:** Multiple virtual machines running on a single hypervisor, adding another layer of abstraction.

| Core OS Goals: 1. Virtualization

Virtualization is the act of making a physical resource appear as multiple logical resources:

- **CPU Virtualization:** The OS takes a single physical CPU and creates the illusion of thousands of independent virtual CPUs (processes).
- **Memory Virtualization:** The OS gives each process its own virtual address space, making it think it has access to the entire RAM range.
- *How:* Achieved through a technique called **Limited Direct Execution (LDE)** (explored in Lecture 2.2).

| Core OS Goals: 2. Concurrency

Operating Systems must manage many tasks running at the same time:

- Multiple threads within the same program.
- Multiple application processes running on different CPU cores.
- Background OS tasks handling device interrupts.

The Challenge: Sharing data safely. If two processes update the same memory location simultaneously without coordination, data gets corrupted. The OS provides locks, semaphores, and synchronization APIs.

| Core OS Goals: 3. Persistence

Computer RAM is volatile — if power is lost, all data vanishes.

Persistence is the ability to store data reliably over the long term.

- **Hardware:** Hard Disks (HDDs), Solid State Drives (SSDs), Optical Media.
- **OS Software Abstraction:** The File System.
- **OS Responsibility:** Ensure file writes are fast, robust against sudden power failures, organized, and access-controlled.

| Core OS Goals: 4. Security & Isolation

In a shared multi-user environment, the OS must enforce strict boundaries:

- **Memory Isolation:** No program should ever write to or read from another program's private memory space.
- **Execution Isolation:** A crashed program should not crash any other running programs or the kernel.
- **Access Control:** Files, network sockets, and devices must have clear ownership and permissions.

| Core OS Goals: 5. Performance & Trade-offs

An operating system must run efficiently:

- **Minimal Overhead:** The OS should not consume too many clock cycles or memory bytes that could otherwise be used by applications.
- **Optimization Trade-offs:**
 - *Throughput vs. Latency:* Do we run batch tasks quickly, or make the user interface responsive?
 - *Security vs. Speed:* Do we double-check every pointer (slower), or trust applications (faster but highly unsafe)?

OS design is a continuous game of balancing trade-offs.

✓ Checkpoint 1 — OS Overview & History

Let's test our understanding:

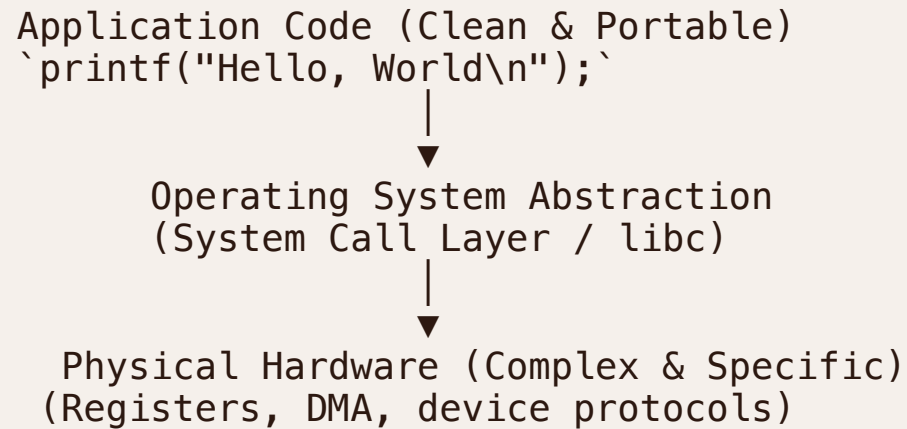
1. Explain the difference between the "Resource Manager" view and the "Virtual Machine" view of an OS.
2. How did the transition from Batch Processing to Multiprogramming improve CPU utilization?
3. What is the fundamental difference between Multiprogramming and Timesharing?
4. Identify the three pillars of operating systems described in OSTEP (*hint: Virtualization is one*).

Write down your thoughts before moving to Section 2.

§ 2 — Core OS Abstractions

| The Power of Abstraction

An **abstraction** is a simplified view of a complex system that exposes essential details while hiding unnecessary mechanics:



The OS provides four fundamental abstractions: Processes, Threads, Address Spaces, and Files.

| CPU Abstraction: The Process

The **Process** is the most important abstraction in operating systems.

- **Definition:** A process is a running instance of a program in execution.
- **Contents of a Process:**
 - The executable program code.
 - The current state (registers like PC, Stack Pointer).
 - Its private Virtual Address Space.
 - Opened file descriptors and I/O sockets.
- *Virtualization:* The CPU switches between processes so fast that each process believes it has dedicated CPU hardware.

| Program vs. Process

A common point of confusion:

Aspect	Program	Process
State	Passive entity	Active entity
Location	Binary file stored on Disk (ELF, EXE)	Active allocation in RAM
Lifetime	Permanent (until deleted)	Temporary (from execution to termination)
Resources	Consumes only disk space	Consumes CPU cycles, RAM, files, sockets

Analogy: A program is a recipe in a book; a process is a chef active in the kitchen cooking the meal.

| CPU Abstraction: The Thread

Within a single process, there can be multiple execution flows called **Threads**:

- A thread is the smallest unit of execution schedulable by the OS.
- All threads inside a process share:
 - The same address space (code and global data).
 - Open files and descriptors.
- However, each thread has its own:
 - Private Program Counter (PC).
 - Private Stack (for local variables and frames).
 - Register state.

Thread vs. Process Memory Layout

Single-threaded Process

Code
Static Data
Heap
Stack

Multi-threaded Process

Code		
Static Data		
Heap		
Stack 1	Stack 2	Stack 3

- In a multi-threaded process, the Heap and static data segments are fully shared.
- Each thread has its own private Stack segment to maintain local variable states.

| Memory Abstraction: The Address Space

Physical RAM is a shared grid of cells. The OS abstracts this into an **Address Space**:

- Each process is given a contiguous range of addresses (typically from `0` to $2^{64} - 1$ in 64-bit systems).
- This address space is **Virtual**:
 - Virtual address `0x1000` in Process A maps to physical RAM address `0x7fff000`.
 - Virtual address `0x1000` in Process B maps to physical RAM address `0x8aaa000`.
- Process A cannot see or access Process B's memory, ensuring complete safety.

| Why Virtualize Memory?

Memory virtualization provides three key benefits:

1. **Ease of Use:** Programmers do not need to worry about where their code is loaded in physical RAM. They can compile code assuming it starts at address `0`.
2. **Isolation:** If Process A runs wild and dereferences a bad pointer (e.g. `NULL`), it only crashes itself. It cannot corrupt other processes or the kernel.
3. **Efficiency:** The OS can load only parts of a program into memory (Paging), swapping unused sections to disk to save RAM.

| Storage Abstraction: The File

Secondary storage (HDDs, SSDs) operates on physical blocks and sectors (e.g., read block 512).

The OS abstracts this into **Files**:

- A file is a named, linear array of bytes that can be read or written.
- The user does not care which sectors on the SSD contain the file data, or if the file is fragmented across the physical drive.
- The OS manages the mapping between file bytes and physical drive blocks.

| Storage Abstraction: Directories

To organize files, the OS provides the **Directory** abstraction:

- A directory is a special file that contains a list of human-readable names and pointers to other files or directories.
- This creates a **Hierarchical Directory Tree**:

```
/ (Root)
├── bin/
│   └── ls
├── usr/
└── home/
    ├── gabriele/
    │   └── os-course/
```

- The OS translates paths (like `/home/gabriele/file.txt`) into physical drive coordinates.

| Network Abstraction: Sockets

Network cards transmit and receive raw frames over wires or radio waves.

The OS abstracts this network interface using **Sockets**:

- A socket is a logical endpoint for data communication.
- Programmers open a socket, connect it to an IP address and port, and write bytes to it using standard file-like descriptors.
- The OS kernel handles packet formatting, TCP handshakes, window sizing, routing, and physical transmission under the hood.

| Core Abstractions Summary Table

Here is how the OS abstracts physical reality:

Physical Resource	OS Abstraction	Key Operations
CPU Core	Process / Thread	<code>fork()</code> , <code>exec()</code> , <code>pthread_create()</code>
Physical RAM	Virtual Address Space	<code>malloc()</code> , <code>mmap()</code> , pointer dereferences
HDD/SSD Sectors	File & Directory	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>mkdir()</code>
Network Interface	Socket	<code>socket()</code> , <code>connect()</code> , <code>send()</code> , <code>recv()</code>

✓ Checkpoint 2 — Core OS Abstractions

Let's test our understanding:

1. Contrast a **Process** with a **Thread**. What resources do they share, and what do they keep private?
2. If Process A and Process B both read pointer address `0x4016a0` simultaneously, do they read the same physical memory cell? Explain.
3. What physical resource does a **Socket** abstract?
4. Why is the "Recipe vs. Chef" analogy useful for explaining Programs vs. Processes?

Think about these abstractions before moving to the hardware level.

§ 3 — Hardware Support & Dual-Mode Execution

| Why the OS Needs Hardware Assistance

The Operating System is just software. If it runs on the CPU exactly like a user application, what prevents a user program from:

- Writing code that disables the OS memory protection?
- Reading personal files belonging to another user?
- Disabling interrupts and hogging the CPU forever?

Answer: Nothing. Software alone cannot enforce boundaries. The OS requires **Hardware Support** built into the CPU to police execution.

| The Anarchy Problem

Without CPU-enforced boundaries, buggy or malicious programs could easily break the entire system:

Buggy User Program
Writes to address 0x000000 (NULL)
If direct RAM access allowed -> overwrites OS code!



Kernel Code Collapses
SYSTEM CRASH

Therefore, the hardware must restrict what user programs can do.

| Dual-Mode Execution

Modern CPUs provide at least two distinct modes of operation:

1. User Mode (Non-Privileged)

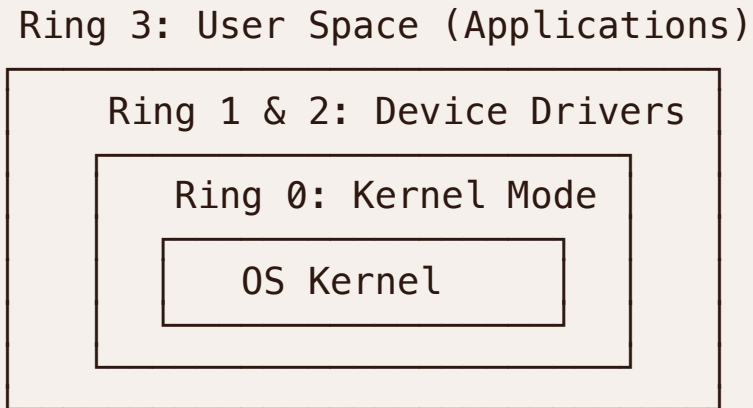
- Used for running user applications.
- Access is restricted. The CPU will refuse to execute dangerous instructions or access unauthorized memory addresses directly.

2. Kernel Mode (Privileged / Supervisor)

- Used for running the OS Kernel code.
- Full access to all CPU instructions and all physical hardware registers.

Protection Rings (x86-64 Architecture)

On x86 architectures, there are four privilege levels called **Rings**:



- In practice, modern operating systems like Linux and Windows use only **Ring 0** (Kernel Mode) and **Ring 3** (User Mode).

| Ring Isolation in Modern Systems

Why do OSes only use Ring 0 and Ring 3?

- **Simplicity:** Two rings are enough to enforce the basic boundary between trusted operating system code and untrusted user programs.
- **Portability:** Many CPU architectures (like ARM or MIPS) only support two privilege modes. Implementing a 4-ring design makes porting the OS to other architectures difficult.
- Ring 1 and Ring 2 were historically intended for device drivers, but in modern OSes, device drivers run inside the Ring 0 kernel for speed, or Ring 3 for safety.

| Implementing Modes: The Mode Bit

How does the CPU know which mode it is currently in?

- The CPU utilizes a **Mode Bit** stored inside a status register (e.g., the **PSR** or the Code Segment Selector register in x86).
- When the Mode Bit is **0**, the CPU runs in **Kernel Mode** (full privileges).
- When the Mode Bit is **1**, the CPU runs in **User Mode** (restricted privileges).
- Every instruction decoded by the Control Unit is checked against the current value of the Mode Bit.

| Privileged Instructions

Privileged instructions are operations that can only be executed when the CPU is running in **Kernel Mode** (Mode Bit = 0).

Examples of Privileged Instructions:

- **HLT (Halt)**: Turns off or sleeps the CPU.
- **CLI** / **STI**: Disable/Enable hardware interrupts.
- **Modifying Page Table Registers**: Adjusting the MMU pointer to change memory maps.
- **Direct I/O Instructions**: Sending control signals to hardware registers (**in** / **out** on x86).

| Non-Privileged Instructions

Non-privileged instructions are safe operations that any program can execute at any time (even in User Mode):

- **Arithmetic:** `add`, `sub`, `mul`, `div`.
- **Logic:** `and`, `or`, `xor`, shifts.
- **Data Movement:** Reading or writing registers, loading from or storing to virtual memory addresses that are allocated to the process.
- **Control Flow:** `jmp`, `cmp`, conditional jumps, calling functions within the program's code space.

| Privileged vs. Non-Privileged Checklist

Let's look at common instruction tasks and their required mode:

Instruction Task	Allowed in User Mode?	CPU Mode Required
Add two registers	✓ Yes	User or Kernel
Read system timer clock	✓ Yes	User or Kernel
Change MMU page table base	✗ No	Kernel Mode Only
Read byte from I/O port	✗ No	Kernel Mode Only
Direct write to physical RAM block	✗ No	Kernel Mode Only
Clear the Interrupt Enable Flag	✗ No	Kernel Mode Only

| Transitioning from User to Kernel Mode

User programs cannot switch to Kernel Mode by writing to the Mode Bit directly (that would be a privileged operation!).

The **only** ways to switch from User to Kernel Mode are through hardware-controlled entry points:

1. **System Calls / Traps:** User program explicitly requests an OS service.
2. **Exceptions:** User program does something illegal (division by zero, memory access violation).
3. **Hardware Interrupts:** External device pulls an interrupt line active (e.g., timer expires, packet arrives).

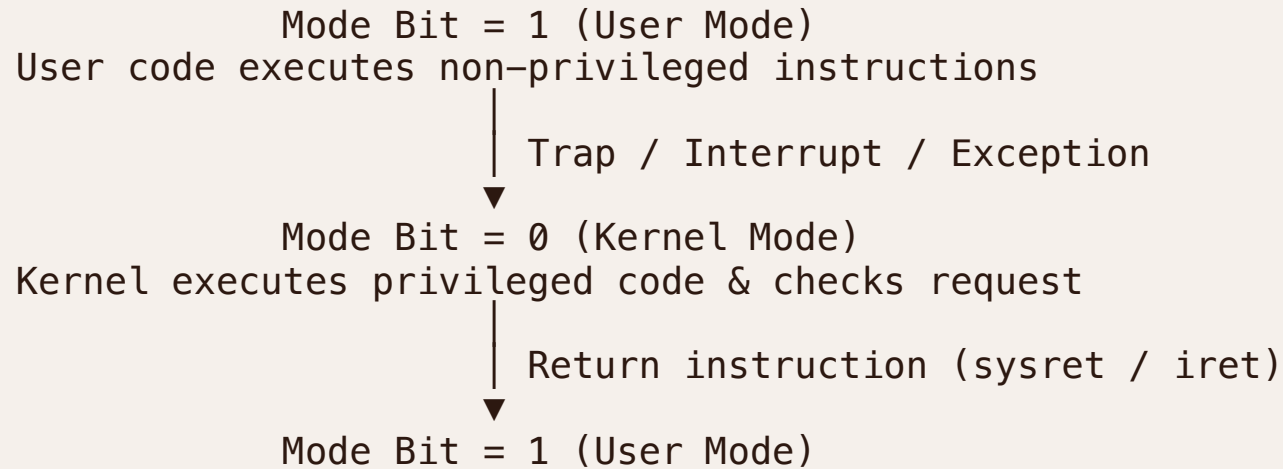
| Transitioning from Kernel to User Mode

To resume running user applications safely, the kernel must switch back to User Mode:

- The kernel prepares the user context (registers, stack pointer).
- It executes a special instruction (e.g., `sysret`, `sysexit` or `iret` on x86).
- This instruction atomically:
 - i. Restores the user Program Counter (PC).
 - ii. Sets the CPU Mode Bit back to `1` (User Mode).
 - iii. Resumes execution at the user address.

Atomicity is critical: if we switched modes before jumping, the CPU would execute user instructions in kernel mode!

Hardware Enforcement Summary



✓ Checkpoint 3 — Dual-Mode Execution

Let's test our understanding:

1. What prevents a user program from executing `cli` (disable interrupts) to monopolize the CPU?
2. What is the **Mode Bit**, and where is it physically stored?
3. Explain the term **privileged instruction** and give three concrete examples.
4. Why must the switch from Kernel Mode to User Mode happen **atomically** with the jump back to user code?

Ensure you understand Ring 0 vs. Ring 3 before proceeding.

§ 4 — The System Call Interface (Syscalls)

| The Challenge of Safe Services

User applications need services that require privileges:

- Writing data to files (accessing disk controller).
- Sending network packets (accessing network card).
- Creating new processes.

The Dilemma: If user apps cannot run privileged instructions, how do they write files or send packets?

The Solution: The **System Call (Syscall)** interface. The app asks the kernel to perform the action on its behalf.

| System Calls vs. Standard Library Calls

Not all function calls are system calls:

- **Library Call (e.g., `strlen()`):** Executed entirely within User Mode. Just a standard jump instruction to memory within the process code segment. Very fast.
- **System Call (e.g., `open()`):** Invokes the kernel. Triggers a hardware transition to Kernel Mode, executes OS code, and returns to User Mode. Significantly slower.

Note: Standard library functions like C's `printf()` or `fopen()` are wrappers. They do user-space bookkeeping, then call `write()` or `open()` system calls.

| The Role of libc Wrapper Functions

Most programmers do not write raw assembly system calls directly. They use libraries like `glibc` in C:

```
User App:  calls wrapper `write(fd, msg, size)`  
           | (Standard function call)  
           ▼  
glibc code: Sets registers with arguments & Syscall ID  
           Executes TRAP / SYSCALL instruction  
           | (Hardware transitions to Ring 0)  
           ▼  
Kernel:    System Call Handler executes actual write
```

This wrapper design shields developers from low-level assembly syntax and hardware-specific system call registers.

| Detailed Syscall Lifecycle

What happens when an application calls `read()` :

1. **Setup:** The library wrapper copies arguments (file descriptor, buffer address, size) to specific CPU registers (e.g., `%rdi`, `%rsi`, `%rdx` on x86-64).
2. **ID:** The unique System Call Number (e.g., `0` for read on Linux x86-64) is loaded into `%rax`.
3. **Trap:** The wrapper executes the `syscall` (or `int 0x80`) instruction.
4. **Transition:** The CPU checks the syscall vector, switches Mode Bit to `0`, and jumps to the kernel's central syscall handler address.
5. **Execution:** The kernel validates arguments, performs the I/O, places the return value in `%rax`, and executes `sysret`.
6. **Return:** The CPU switches Mode Bit to `1` and resumes the application.

Syscall Assembly Flow Diagram

User Space (Ring 3)

```
mov  eax, 1      ; write ID
mov  edi, 1      ; stdout fd
mov  rsi, msg_ptr ; message address
mov  edx, 14     ; length
syscall          ; — Trap —>
```

Kernel Space (Ring 0)

```
Switch to Mode 0
Read EAX (ID = 1)
Lookup in syscall_table[1]
Jump to sys_write()
Execute write to screen
Set return val in EAX
sysret (Switch to Mode 1)
```

resumes here ←
eax contains bytes written

| System Call Translation Tables

How the kernel links numbers to actual functions:

- The kernel source code defines a table of function pointers:

```
// Simplified Linux Syscall table
void *sys_call_table[400] = {
    [0] = sys_read,
    [1] = sys_write,
    [2] = sys_open,
    [3] = sys_close,
    // ...
};
```

- The assembly trap handler simply reads the index stored in register `EAX`, verifies it is < 400 , and runs `call sys_call_table[eax]`.

| How are Arguments Passed to the Kernel?

How do syscall parameters cross the user-kernel boundary? Three methods:

1. **Registers:** Arguments are loaded directly into designated registers (fast, but limited to a few parameters).
2. **Stack / Block Pointer:** If parameters exceed registers, the wrapper stores them in a memory block and passes a single register pointer pointing to the block.
3. **User Stack:** Arguments are pushed onto the user stack, and the kernel accesses the user stack pointer to read them.

| Kernel Rule #1: Never Trust User Input!

The kernel runs with full hardware privileges. A bug in the kernel can crash the machine or expose all system data.

When processing a system call, the kernel must validate all parameters:

- **Pointers:** Does the user-supplied pointer point to valid user memory? If a user passes a pointer pointing to kernel memory, the kernel must reject it!
- **Boundaries:** Are buffer sizes valid? (Preventing buffer overflows).
- **Permissions:** Does this process have access rights to read this specific file?

| Common POSIX System Calls

Modern UNIX-like operating systems share a standard set of system calls:

Area	Syscall	Description
Process Control	<code>fork()</code> , <code>exec()</code> , <code>wait()</code> , <code>exit()</code>	Manage process lifecycle
File Management	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>	Standard file I/O operations
Device Control	<code>ioctl()</code>	Configure device parameters
Information	<code>getpid()</code> , <code>alarm()</code>	Query status, set timers
Communication	<code>pipe()</code> , <code>shmget()</code>	Inter-process communication

| Why are System Calls Expensive?

System calls are significantly slower than standard function calls due to hardware and software overheads:

- **Privilege Transition:** Changing Ring 3 $\rightarrow$ Ring 0 $\rightarrow$ Ring 3.
- **Register Saving:** The CPU and kernel must save all user registers to the stack so they are not clobbered by kernel execution.
- **Cache Pollution:** The kernel executes different code and accesses different data, filling the L1/L2 caches and TLB with entries that displace the application's cached data.
- **Context Switch:** If the system call triggers a block (e.g. reading from slow disk), the OS will schedule another process, forcing a full context switch.

| Minimizing Syscall Frequency: Buffering

To keep performance high, software libraries try to minimize syscalls:

- **Example: `printf` vs. `write`**
 - If you call `printf` 100 times to print 1 character each, calling `write` 100 times would be extremely slow.
 - The standard library (`stdio.h`) buffers the characters in a user-space memory buffer.
 - When the buffer is full or a newline `\n` is detected, the library makes a **single** system call containing all 100 characters.
 - This reduces the system call overhead by 100x!

✓ Checkpoint 4 — System Call Interface

Let's test our understanding:

1. Why can't we use a standard assembly `jmp` instruction to execute kernel space functions?
2. What are the roles of the registers `%rax` (on x86-64) and the `syscall` instruction during a system call?
3. What is "Argument Validation" and what security threat does it prevent?
4. How do standard library buffering mechanisms (like `printf` buffering) minimize system call overhead?

Solve these before moving to Section 5.

§ 5 — Interrupts, Exceptions & Traps

| Handling Unexpected Events

During normal sequential program execution, events occur that require immediate attention:

- A user hits a key on the keyboard.
- A program attempts to divide a number by zero.
- An application requests a file read from the disk.

The operating system handles all these events using a unified mechanism: **Processor Interrupts**.

| Traps, Exceptions, and Interrupts

These events are classified by their source and synchronization:

Event Type	Source	Synchronous?	Intentional?	Example
Trap / Syscall	Software	Yes	Yes	Requesting file read
Exception	CPU Fault	Yes	No	Div-by-zero, page fault
Interrupt	Hardware	No (Asynchronous)	No	Keyboard key press

- **Synchronous:** Happens at a specific instruction. Re-running the code with same inputs triggers it at the exact same point.
- **Asynchronous:** Can happen at any time, unrelated to the current instruction.

| 1. Traps (Software Interrupts)

- **Definition:** An intentional, synchronous exception generated by executing a specific instruction (like `syscall` or `int`).
- **Use Case:** Used exclusively to implement the System Call interface, allowing user programs to enter the kernel controlled entry point.
- **Behavior:** The CPU completes the trap instruction, switches to kernel mode, executes the handler, and returns to the *next* sequential instruction in the user program.

| 2. Exceptions (CPU Faults & Aborts)

Exceptions are generated by the CPU when it detects an error condition while executing an instruction:

- **Faults:** Unintentional but potentially correctable. The CPU handles it, and if corrected, re-executes the offending instruction (e.g. Page Fault).
- **Aborts:** Unrecoverable hardware or system error. The kernel immediately terminates the process (e.g. Machine Check Exception).

Divide by Zero → CPU raises Exception → Kernel halts process

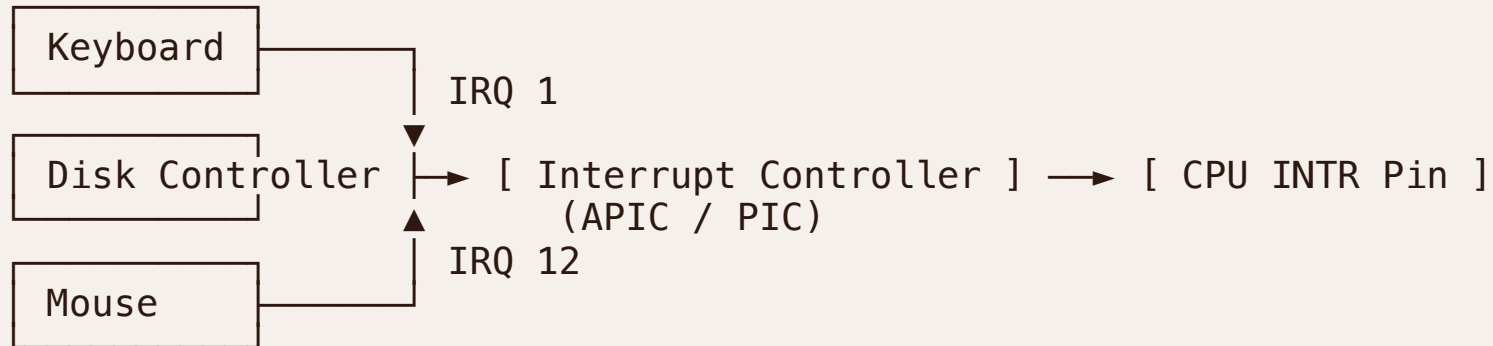
| 3. Interrupts (Hardware Interrupts)

Interrupts are signals sent to the CPU by external hardware devices:

- **Asynchronous:** They are triggered by external physical events (e.g., network packet arrives on NIC, timer clock tick fires). They have no relationship to the instruction currently executing.
- **Behavior:** The CPU completes its *current* instruction, handles the interrupt, and then returns to the *exact same* instruction that was about to run next.
- Allows the CPU to run computation continuously without wasting cycles checking (polling) devices.

| The APIC (Advanced Programmable Interrupt Controller)

External devices cannot pull the CPU pins directly. They route through an Interrupt Controller:



- The APIC manages hardware priorities, masking (disabling low-priority interrupts), and routing signals to specific CPU cores.

| The Interrupt Descriptor Table (IDT)

Every event is identified by a **vector number** (0–255) that indexes a table of handlers. Two generations of this table exist:

- **IVT** (*Interrupt Vector Table*): 8086 real-mode. 256 × 4-byte `segment:offset` pointers at fixed address `0x0`.
No privilege checks.
- **IDT** (*Interrupt Descriptor Table*): protected/long-mode successor used by modern OSes. Entries are 16-byte **gate descriptors**; the table lives anywhere in RAM, located via the `IDTR` register (loaded by the OS at boot with `lidt`).

Each gate descriptor holds: the **handler address**; the **code segment** (Ring 0); the **DPL** (highest ring allowed to invoke it via `int n`; hardware events ignore it); and the **gate type**:

Gate type	On entry, the CPU...	Typical use
Interrupt Gate	clears <code>IF</code> (interrupts off)	Hardware interrupts, most exceptions
Trap Gate	leaves <code>IF</code> unchanged	Software traps (<code>int 0x80</code>), some exceptions

Event type (what happened) ≠ *gate type* (how the CPU enters the handler): exceptions have no gate of their own; the OS picks one of the two per vector.

| IDT Layout

A typical x86 vector allocation:

Vector	Event type	Description / Handler	Gate
0	Exception (fault)	Divide-by-zero	Interrupt/Trap
13	Exception (fault)	General Protection Fault	Interrupt/Trap
14	Exception (fault)	Page Fault	Interrupt
32	Hardware interrupt	System timer tick	Interrupt
33	Hardware interrupt	Keyboard	Interrupt
128 (0x80)	Software trap	Legacy Linux syscall (<code>int 0x80</code>)	Trap, DPL=3

- Vectors **0–31** are reserved for CPU exceptions; **32–255** serve hardware interrupts and software traps.
- Only vector `0x80` has `DPL=3`, so user code can invoke it; a user-issued `int 14` raises a GPF instead.
- The modern `syscall` instruction bypasses the IDT (target address kept in a dedicated **Model-Specific Register**).

| Detailed Step-by-Step Interrupt Handling

What the CPU hardware does automatically when an interrupt line is pulled:

1. **Finish:** Completes execution of the current instruction.
2. **Save:** Pushes the current status registers (`EFLAGS / RFLAGS`), Program Counter (`CS:RIP`), and Stack Pointer (`SS:RSP`) onto the target kernel stack.
3. **Switch:** Switches the Mode Bit to `0` (Kernel Mode).
4. **Vector:** Obtains the vector number (from the interrupt controller for hardware interrupts; fixed by the CPU for exceptions; the operand of `int n` for software traps), indexes the IDT via `IDTR` , and loads the handler's address into the PC.
5. **Execute:** The CPU begins executing the ISR.

| Inside the ISR (Software Phase)

Once the CPU jumps to the ISR:

1. **Save Rest:** The ISR code (written in assembly/C) pushes any other registers it will use onto the kernel stack (Registers not saved by hardware).
2. **Process:** Handles the device request (e.g., read keycode from keyboard controller data register).
3. **Acknowledge:** Sends an End of Interrupt (EOI) signal to the APIC, allowing the controller to send new interrupts.
4. **Restore:** Pops the saved registers.
5. **Return:** Executes `iret` to return to the user program.

| Disabling Interrupts

Sometimes the kernel must execute critical code without being interrupted (e.g., updating scheduler lists):

- The CPU can disable interrupts using special instructions:
 - **CLI** (Clear Interrupt Flag) → CPU ignores external interrupts.
 - **STI** (Set Interrupt Flag) → CPU accepts interrupts.
- **Safety:** **CLI** and **STI** are strictly **privileged instructions**.
- If a user application could run **CLI**, it could monopolize the CPU forever, rendering the OS scheduler helpless.

| Maskable vs. Non-Maskable Interrupts

Not all interrupts can be ignored:

Maskable Interrupts

- Standard hardware interrupts (timer, keyboard, disk).
- Can be temporarily disabled by the CPU using the `CLI` instruction.

Non-Maskable Interrupts (NMI)

- High-priority, critical hardware events (e.g., memory parity error, power failure warning).
- Cannot be disabled by the CPU. The CPU must process the NMI immediately.

| The Timer Interrupt

The **Timer Interrupt** is a crucial piece of hardware support:

- A hardware clock generator is configured to interrupt the CPU periodically (e.g., every 1 to 10 milliseconds).
- **The Role:** It forces control back to the Operating System kernel.
- **Preemption:** Without timer interrupts, a cooperative program containing an infinite loop `while(1) {}` would run forever and lock up the computer.
- The timer interrupt handler automatically triggers the OS scheduler to check if the current process should be switched.

✓ Checkpoint 5 — Interrupts, Exceptions & Traps

Let's test our understanding:

1. Create a table comparing Traps, Exceptions, and Interrupts on: source, synchronization, and intent.
2. What would happen if the CPU did not push the user registers and flags onto the stack before executing an ISR?
3. Why are `cli` and `sti` instructions privileged?
4. What is the difference between a maskable and a non-maskable interrupt?
5. How does the **Timer Interrupt** enable preemption?

Review your answers before proceeding.

§ 6 — Memory Protection & DMA Coordination

Virtual Address Range Boundaries

On modern 64-bit systems, the virtual address space is divided between user and kernel:

0xFFFF800000000000	Kernel Space Memory	(Ring 0 only)
0xFFFF800000000000	Unmapped	(Canonical address gap)
0x00007FFFFFFFFFFF	User Space Memory	(Ring 3 / Applications)
0x0000000000000000		

- Any attempt by user-space code to access addresses above **0x00007FFFFFFFFFFF** immediately triggers a hardware exception.

| Basic Memory Protection

To isolate processes, the CPU must check every memory access.

The simplest hardware protection mechanism is the **Base and Bounds Registers** scheme:

- **Base Register:** Holds the starting physical address of the process.
- **Bounds Register:** Holds the size/limit of the allocated memory range.

Physical RAM:

Kernel Space
Process A Address Space
Process B Address

← Base Register (0x100000)

← Bounds Register (Limit: 0x050000)

| Base and Bounds Translation

For every memory reference generated by a user program, the CPU hardware performs translation and validation:

$$\text{Physical Address} = \text{Virtual Address} + \text{Base}$$

Validation Check:

$$\text{Virtual Address} \geq 0 \quad \text{and} \quad \text{Virtual Address} < \text{Bounds}$$

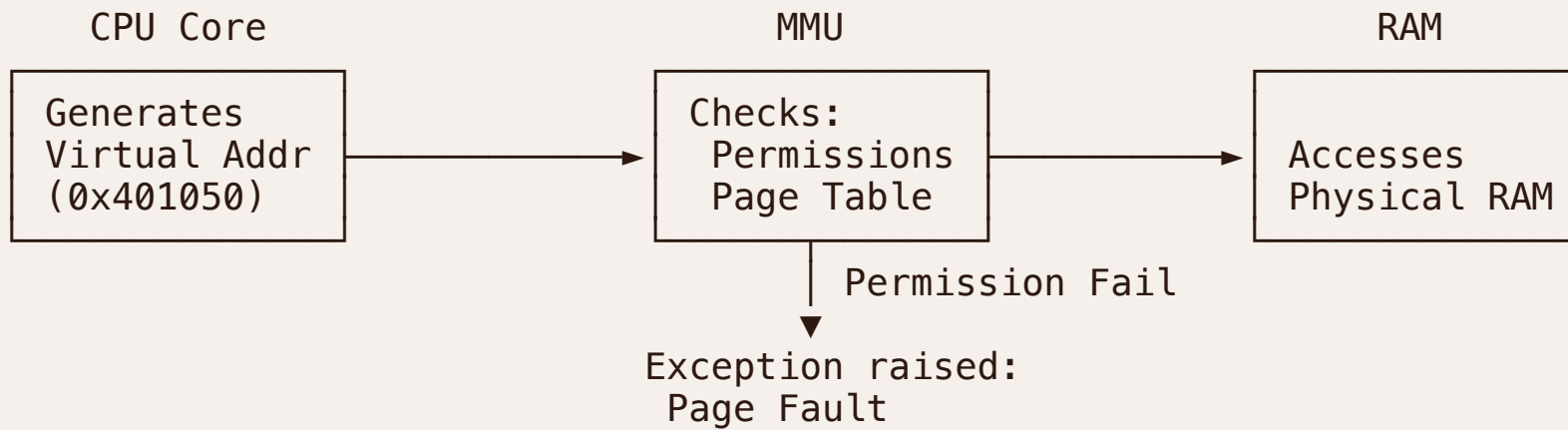
- If the validation check fails, the MMU blocks access and raises a **Segment Violation Exception** (Segmentation Fault).
- **Security:** Setting Base and Bounds registers is a **privileged instruction**. Only the kernel can modify them during a context switch.

| The Memory Management Unit (MMU)

Modern systems use virtual memory with **Paging** rather than simple base/bounds:

- The **MMU** is a hardware component in the CPU that translates virtual addresses to physical addresses on the fly using Page Tables.
- Each page table entry (PTE) contains permission flags:
 - **Read/Write (R/W) Bit:** If 0, memory is read-only (e.g. code segment).
 - **User/Supervisor (U/S) Bit:** If 0, only kernel mode can access it.
 - **No-Execute (NX / XD) Bit:** Prevents running code from this page (stops buffer overflow exploits in stack).

MMU Address Translation



| The Page Fault Exception

What happens when translation fails or permissions are violated?

- The MMU triggers a **Page Fault Exception** (Vector 14).
- The CPU switches to Kernel Mode and executes the page fault handler.
- **Possible Kernel Responses:**
 - *Valid page, but swapped out:* Kernel loads the page from disk into RAM, updates page table, and restarts the instruction (transparent to program).
 - *Invalid access (e.g., writing to read-only code):* Kernel sends **SIGSEGV** and terminates the program.

| DMA and the Memory Protection Conflict

In Lecture 1.1, we saw that the DMA Controller bypasses the CPU to read/write directly to RAM.

This introduces a security risk:

- Can a user program configure the DMA controller to write data directly into the kernel's memory space?

Hardware Protection:

- The registers of the DMA controller are mapped to privileged I/O ports or MMIO addresses.
- Because these addresses are marked as "Supervisor Only" in the page tables, user programs cannot write to them. Only the kernel can configure DMA transfers.

| DMA and Physical Addressing

Another challenge with DMA:

- User programs use **Virtual Addresses**.
- The DMA controller communicates directly with the physical memory bus, using **Physical Addresses**.
- **OS Protocol:**
 - The user program passes a virtual address to the kernel via a syscall (e.g. `write()`).
 - The kernel translates this virtual address into a physical address.
 - The kernel writes the physical address to the DMA controller's registers.
 - The DMA transfer operates safely.

| I/O Protection & Memory-Mapped I/O (MMIO)

How does the hardware prevent direct access to I/O controllers?

- On systems using Port-Mapped I/O, CPU instructions like `in` and `out` check the current execution ring. If $\text{Ring} > 0$, they fail.
- On systems using Memory-Mapped I/O (MMIO), the physical registers of device controllers are mapped to specific physical addresses.
- The kernel sets up the page tables so that these MMIO physical ranges are marked as **Supervisor Only** (accessible only to Ring 0).
- User processes cannot write to these addresses; they must route via the kernel device driver system calls.

✓ Checkpoint 6 — Memory Protection & DMA

Let's test our understanding:

1. If Base = `0x200000` and Bounds/Limit = `0x10000`, what physical address corresponds to virtual address `0x0050`? What happens if the program accesses virtual address `0x12000`?
2. What are the roles of the R/W, U/S, and NX bits in a Page Table Entry?
3. How does the OS protect the DMA controller from being hijacked by a user-space application?
4. Why must the kernel translate user virtual addresses before configuring a DMA transfer?
5. How is MMIO protected from unauthorized user writes?

Consolidate these security concepts.

§ 7 — Summary & Path to Virtualization

| The Hardware/Software Contract

Modern operating systems rely on a symbiotic contract with CPU hardware:

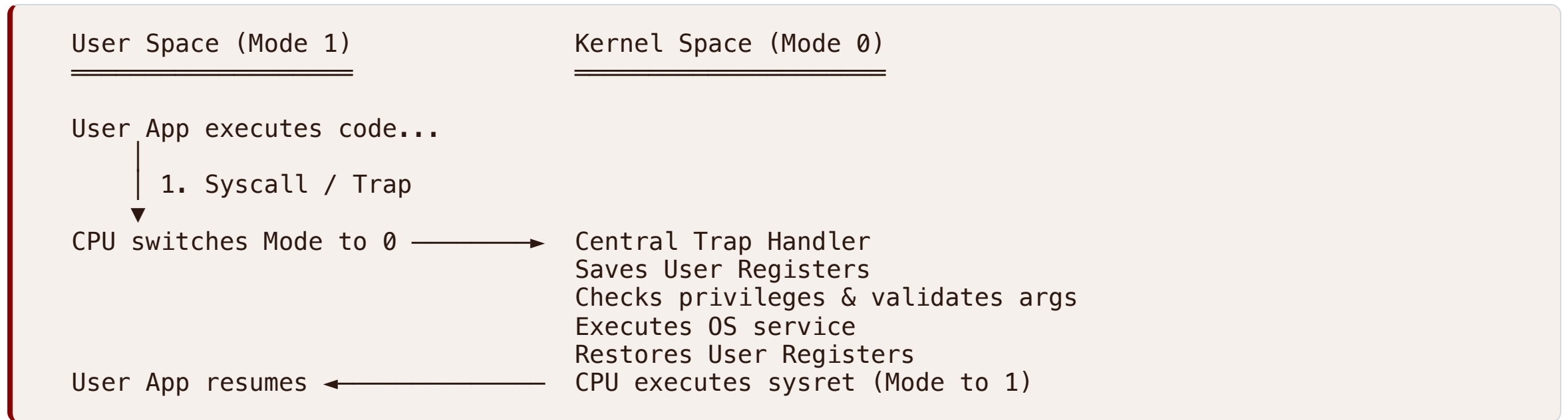
Hardware Support Feature	OS Capability Enabled
Dual-Mode (Ring 0 / Ring 3)	Prevents user apps from accessing physical hardware directly.
Privileged Instructions	Ensures only the kernel can configure critical resources.
MMU & Page Tables	Creates virtual address spaces, isolating processes.
Traps & Syscall Instruction	Provides a secure gate for apps to request kernel services.
Interrupts & Timer Clock	Allows scheduler to preempt running apps (Timesharing).

| Interrupt Comparison: HW vs. SW

Key differences:

Property	Hardware Interrupt	Software Interrupt (Trap)
Trigger	Electrical signal on CPU pin	Executing an instruction (<code>int 0x80</code> , <code>syscall</code>)
Sync	Asynchronous (any time)	Synchronous (predictable execution)
Maskability	Can be disabled by CPU (<code>cli</code>)	Cannot be masked or disabled
Priority	Managed by APIC controller	High CPU priority (handles fault/trap)

| The Complete Execution Picture



| Practice Exam Question 1

Question: If a program runs in user mode and attempts to execute the **HLT** (halt) instruction, what happens step-by-step?

Answer:

1. The CPU Control Unit decodes **HLT** and checks the current Mode Bit.
2. Mode Bit is 1 (User Mode), which does not permit **HLT** (it is a privileged instruction).
3. The CPU triggers a General Protection Exception.
4. Mode is switched to 0, and the CPU jumps to the exception handler address.
5. The kernel identifies the error, terminates the process, and prints a diagnostic crash message.

| Practice Exam Question 2

Question: Explain why context switching would be impossible without a Timer Interrupt.

Answer:

- Without a timer interrupt, the CPU executes instruction after instruction of the running process.
- The only way the OS kernel can regain control is if the process voluntarily relinquishes it (via a syscall or yield) or crashes.
- If a program enters a loop `while(1) {}`, it will run forever, and the scheduler will never get CPU time to run a context switch.
- The timer interrupt periodically preempts the program automatically.

| Summary

We have established the hardware/software contract:

- **Lecture 1.1:** Explored the structural pillars (CPU, Memory, I/O) and FDE.
- **Lecture 1.2:** Studied how the OS acts as a manager, leveraging dual-mode, syscalls, and interrupts to govern execution.

| Lecture 1.2 Complete

Recommended Reading

- **OSTEP:** Chapter 2 (Introduction to Operating Systems)
- **Silberschatz:** Chapter 1 and 2 (OS Structures & Hardware Support)

Exercises to Prepare for Exams

- Walk through the steps of a page fault exception. What registers are saved, and who restores them?
- List three actions that a user-space application *cannot* perform directly.

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!